

- If $y - \overline{\text{next}}(X_{a,k})$ and $z - \overline{\text{next}}(X_{a,k})$ have different signs, then according to Lemma C.19, we have, $y = \text{ROUND}(\text{next}(X_{a,k}), p)$ because S_k takes consecutive and disjoint values in $\text{ROUND}(1/\lceil n^A \rceil, p)$. This then implies that

$$\begin{aligned} & (y - \overline{\text{next}}(X_{a,k}))^2 - (z - \overline{\text{next}}(X_{a,k}))^2 \\ & \leq O\left(\frac{1}{n^{10A}}\right) + \frac{1}{\text{next}^6(X_{a,k})} - \left(\frac{1}{\text{next}(X_{a,k})(\text{next}(X_{a,k}) + 1)}\right)^2 \\ & \leq -\frac{1}{4n^{4A}}. \end{aligned}$$

Concluding, we always have for any $k'' \neq k^* = \arg \max_{k',k} \left((W^{(K)} X)^\top (W^{(Q)} X) \right)_{k',k}$

$$\left((W^{(K)} X)^\top (W^{(Q)} X) \right)_{k',k} - \left((W^{(K)} X)^\top (W^{(Q)} X) \right)_{k^*,k} \leq -\frac{n}{4}.$$

Concluding the above discussion, we have after the column-wise softmax and rounding to $p = O(\log n)$ bit,

$$\left(\text{softmax} \left((W^{(K)} X)^\top (W^{(Q)} X) \right) \right)_{k',k} = \mathbf{1} \left[k' = \arg \min_{h \in \{h | X_{I_1,h} = X_{I_2,k}\} \cup \{1\}} |X_{a,h} - \text{next}(X_{a,k})| \right].$$

Further, we will choose $W^{(V)}$ such that

$$\begin{aligned} W^{(V)} X'_{I_3,k'} &= X'_{I_4,k'} = X_{I_4,k'} \quad \forall k' \in [l]. \\ W^{(V)} X'_{I_3,k'} &= 0 \quad \forall k' \in [l]. \end{aligned}$$

This then concludes the proof. $\square$

C.3 Building Blocks of RNNs Construction

We will now describe the building blocks of Linear RNNs construction. We will introduce some basic operations that will be used to build more complex RNNs family.

Lemma C.20 (Recent Input Memorizing). *Given any constant k and C , there exists a parameter configuration of linear unit that maintains C dimensions of last k input vectors in the state.*

Proof. Suppose the input sequence is $x_{1:t} \in \mathbb{R}^d$, and the dimensions that the state should memorize are $d_1, d_2, \dots, d_C$. We can construct the following linear unit:

$$h_t = [x_{t-1,d_1} \quad \dots \quad x_{t-1,d_C} \quad h_{t-1,1} \quad \dots \quad h_{t-1,C \times (k-1)} \quad h_{t-1,C \times k+1} \quad \dots \quad h_{t-1,d}].$$

$\square$

Lemma C.21 (Summation). *Given any constant k and C , there exists a parameter configuration of linear unit that maintains the sum of one dimension of the last k input vectors in the state.*

Proof. Suppose WLOG the input sequence is $x_{1:t} \in \mathbb{R}^d$, and the dimension that the state should memorize is 1. We can construct the following linear unit:

$$h_t = [x_{t-1,1} + h_{t-1,1} \quad h_{t-1,2} \quad \dots \quad h_{t-1,d}].$$

$\square$

Lemma C.22 (Special Position Memorizing). *Given any constant k and C , there exists a parameter configuration of linear unit and a FFN with Ck width that maintains the C dimensions of the input vector at position 1 to k in the state.*

Proof. This is a direct combination of Lemma C.21 and Lemmas C.1 and C.4. The FFN can assign all the input vectors with position greater than k to 0, and permute the corresponding dimensions of first k input vectors to the first Ck dimensions of the state. The linear unit can then maintain the state. $\square$

Lemma C.23 (Recite Fixed Sequence). *Given any constant integer k and C , there exists a FFN with width kC that can output fixed sequence of scalars that takes values in $[C]$ on a fixed set of positions $l_1, \dots, l_k$.*

Proof. This is a direct consequence of Lemma C.4. $\square$

C.4 Proof of Theorem 4.6

We will first restate the theorem for clarity.

Theorem 4.6. *For task $T \in \{\text{Index}, \text{AR}, c\text{-gram retrieval}, \text{Counting}\}$, there exist constant-size Transformers that can solve T . On the other hand, any RNN with $o(n)$ -bit memory cannot solve T of size n with any length of CoT for large enough n .*

Proof. We will discuss by cases.

When T is Index, we will first show why RNN cannot solve the Index question without $\Omega(n)$ memory. The key observation is that the recurrent form of RNNs allowed the algorithm to be run in a streaming fashion with $o(n)$ bit memory. Here streaming means that the algorithm gets to look at each bit of the memory sequentially and can only update a constant size of memory.

Lemma C.24. *Consider the following two-party game, where Alice receives string $x \in \{0, 1\}^n$ and Bob receives an integer k , and Bob wants to know the value of x_k . If only Alice is allowed to send a signal to Bob, then $\Omega(n)$ bit communication complexity is required.*

Proof of Lemma C.24. Suppose there exists a communication protocol where B only receives $o(n)$ bit and can perfectly decide x_k . Because Alice doesn't know k , the protocol must send the same message to Bob for all k . Hence Bob can reconstruct the whole string x with n bit with $o(n)$ bit communication. This is a contradiction. $\square$

Now if RNN can solve the Index problem with $o(n)$ bit memory, then it can also solve the Index problem with $o(n)$ bit communication complexity. This is because Alice can simply run the RNN on input x and send the hidden state to Bob. Then Bob can run the RNN with the hidden state and k to get the answer. This is a contradiction to Lemma C.24. Hence RNN cannot solve the Index problem with $o(n)$ bit memory.

On the other hand, we will show that Transformers can solve the Index problem with $O(\log n)$ bit parameters. This is because using 2 layers of Transformer, we will implement a Match Block (Lemma C.15) that can match the last query token with the position of the previous token and retrieve the type of the matched token to the query token.

When T is AR, without loss of generality, we assume that n is even. The proof is similar to the proof of the proof of the Index problem. As there are n different types of tokens, we can label them as $[n]$. Now for any boolean sequence $x \in \{0, 1\}^{n/2}$, solving AR for the following input is equivalent to solving the Index problem for x :

$$\mathcal{S}_{\text{in}} = \langle s \rangle, 1, x_1 + n/2, 2, x_2 + n/2, \dots, n/2, x_{n/2} + n/2, k$$

This then implies that RNN cannot solve AR with $o(n)$ bit memory. Transformers, on the other hand, can still solve AR with $O(\log n)$ bit parameters, we will use one layer of copying function to copy each token's previous token's type to it. Then we can use the Match Block to match the last query token with the position of the previous token and retrieve the type of the matched token to the query token.

When T is c -gram retrieval, without loss of generality, we assume that n is a multiple of c . The proof is similar to the proof of Theorem 4.6. As there are n different types of tokens, we can label them as $[n]$. Now for any boolean sequence $x \in \{0, 1\}^{n/2}$, solving AR for the following input is equivalent to solving the Index problem for x :

$$\mathcal{S}_{\text{in}} = \langle s \rangle, \underbrace{1, \dots, 1}_{c-1}, x_1 + n/c, \underbrace{2, \dots, 2}_{c-1}, x_2 + n/c, \dots, \underbrace{n/c, \dots, n/c}_{c-1}, x_{n/c} + n/c, \underbrace{k, \dots, k}_{c-1}$$

This then implies that RNN cannot solve c -gram retrieval with $o(n)$ bit memory. Transformers, on the other hand, can still solve c -gram retrieval with $O(\log n)$ bit parameters, we will use one layer of copying function to copy each token's previous $c - 1$ tokens' type to it. Then we can use the Match Block to match the last query token with the position of the previous token and retrieve the type of the matched token to the query token.

When T is Counting, we will first show why RNN cannot solve the Counting question without $\Omega(n)$ memory. Consider the following setup, given any $x \in \{0, 1\}^n$, the input string is $j_1 j_2 \dots j_k$ where $\{j_i \dots j_k\} = \{j \mid x_j = 1\}$, then solving the Counting question for this input string for queried threshold 1 is equivalent to solving the Index problem for x . This then implies that RNN cannot solve the Counting question with $o(n)$ bit memory.

On the other hand, we will show that Transformers can solve the Counting question with $O(\log n)$ bit parameters. This is because using 2 layers of Transformer, we can first use a COPY block to copy the last query token to the token corresponds to the threshold, and then use a COUNT block (Lemma C.14) that can count the number m of the

appearance of the last query token in the input sequence, and then write $1/(m+1)$ to one of the dimension. Finally, we can use the Feed Forward Network on the last layer to multiply threshold +1 with this value and compare the result to 1 to get the answer. $\square$

C.5 Proof of Theorem 4.1

We will first prove a lemma assuming $\text{PSPACE} \not\subseteq \text{P/poly}$.

Lemma C.25. *If $\text{PSPACE} \not\subseteq \text{P/poly}$, then there exists a Turing machine M with linear space complexity that cannot be simulated by a polynomial-size circuit family.*

Proof. We will prove this by contradiction. Assuming for every Turing machine M with linear space complexity, there exists a polynomial-size circuit family $\{C_n\}$ that can simulate M . We will construct a polynomial-size circuit family $\{C'_n\}$ that can decide PSPACE , which contradicts the assumption that $\text{PSPACE} \not\subseteq \text{P/poly}$.

Given any language $L \in \text{PSPACE}$, we can decide L by a Turing machine M_L with space $O(n^k)$ for some constant k . We can consider another language $L' = \{x1^{|x|^k} \mid x \in L\}$. We can decide L' by a Turing machine $M_{L'}$ with linear space complexity by checking the length of the input and then simulating M_L . By the assumption, there exists a polynomial-size circuit family $\{C_n\}$ that can simulate $M_{L'}$. We can then construct a polynomial-size circuit family $\{C'_n\}$ that can decide L by first counting the length of the input and then simulating C_n on the extended input. This contradicts the assumption that $\text{PSPACE} \not\subseteq \text{P/poly}$. $\square$

Now we are ready to prove our theorem.

Theorem 4.1. *Assuming $\text{PSPACE} \not\subseteq \text{P/poly}$, there exists an algorithmic problem such that (1) there exist constant-size Linear RNNs that can solve the problem with polynomial length CoT; and (2) any constant-size regular RNNs cannot solve the problem without CoT.*

Proof. By Lemma C.25, we know that if $\text{PSPACE} \not\subseteq \text{P/poly}$, then there exists a Turing machine M with linear space complexity that cannot be simulated by a polynomial-size circuit family. We will use this result to prove Theorem 4.1.

We design the task as follows, for any n , let $m = \lfloor \log_2 n \rfloor$, for any boolean input x of length m , we will choose input sequence as $\mathcal{S}_{\text{in}} = \overline{0^{n-m}x}$ and the label as $y = \text{YES}$ if $M(x) = 1$ and $y = \text{NO}$ otherwise.

Because we are considering regular RNN with $O(m)$ memory, we know that we can compute the result of RNN without CoT through a circuit family with size $\text{Poly}(m)$. However, we know that M cannot be simulated by a polynomial-size circuit family. Hence, no RNN family with $O(m)$ memory can solve the task for all n .

On the other hand, we can simulate M by the RNN by maintaining the state, the pointer, and the tape of the M inside the state of the RNN. The RNN can then maintain the transition function of the Turing machine in its output function as a lookup table Lemma C.4 and write down the updated state, the direction of the pointer movement, and the updated cell value at the pointer in its context. Paired with the ability to memorize the recent input Lemma C.20, the RNN can then simulate the running of the Turing machine.

Because the space complexity of M is linear in m , the time complexity of M is $\exp(O(m))$ which is polynomial in n . Hence, we can solve the task by an RNN with CoT and $O(m)$ memory and polynomial-size circuit family. $\square$

C.6 Proof of Theorem 4.7

We will now proceed to prove our main theorem, which states that Transformers with chain-of-thought can solve IsTree perfectly, while RNNs cannot. We will first restate the theorem here.

Theorem 4.7. *There exist constant-size Transformers that can solve IsTree with CoT of length $O(n)$. On the other hand, any RNN with $o(n)$ -bit memory cannot solve IsTree with any length of CoT.*

Proof of Theorem 4.7. We will prove this theorem by proving the following lemmas.

Lemma C.26. *For any n and RNN R with $o(n)$ memory, R cannot perfectly solve IsTree of size n .*

Lemma C.27. *There exists a Transformer T with constant depth and width, and $O(\log n)$ precision, that can solve IsTree of size n perfectly with Chain of Thought.*

This proof is a direct combination of Lemmas C.26 and C.27. $\square$